🏗️ AWS Architecture Patterns — Category 12: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Advanced
> **Topics Covered:** Multi-Region Active-Active, Active-Passive, DR Strategies, RTO/RPO, Well-Architected Framework, Event-Driven Architecture, Zero-Downtime Deployments, Blue/Green, Canary

---

📋 Table of Contents

1. 12.1 Multi-Region Active-Active vs Active-Passive
2. 12.2 Disaster Recovery Strategies — RTO/RPO, Pilot Light, Warm Standby
3. 12.3 Well-Architected Framework — 6 Pillars
4. 12.4 Event-Driven Architecture on AWS
5. 12.5 Zero-Downtime Deployments — Blue/Green, Canary on AWS

12.1 Multi-Region Active-Active vs Active-Passive

🟢 What It Is in Simple Terms

Multi-region means running your workload in more than one AWS geographic region simultaneously. Active-Active means ALL regions serve live traffic at the same time. Active-Passive means one region serves all traffic (active) while the other region stands by ready to take over (passive). The choice between them is a fundamental architectural decision that affects cost, complexity, latency, and resilience.

🔍 Why Multi-Region Exists

```
Single-region risks:
├── Entire AWS region goes down (rare but has happened — us-east-1 2021, etc.)
├── Regulatory: "data must not leave EU" → need EU region
├── Latency: users in Asia hitting us-east-1 = 200ms+ round trips
└── Disaster recovery: major incident wipes out your single region

Multi-region solves:
├── Regional AWS failures: other region(s) absorb traffic instantly
├── Global latency: users routed to nearest region
├── Data residency: data stays in its required jurisdiction
└── DR compliance: RTO/RPO requirements mandate multi-region capability
```

🧩 Active-Passive Architecture

```
Active-Passive:
├── ONE region handles ALL production traffic at any time
├── Second region sits idle (or lightly loaded) on standby
├── Failover: if primary fails → DNS cutover → secondary becomes active
└── Failback: after primary recovers → return traffic to primary

┌──────────────────────────────────────────────────────────────┐
│                      Route 53                                 │
│          Primary Record: us-east-1 (weight 100)               │
│          Failover Record: eu-west-1 (Health Check → failover)│
└──────────────────────────────────────────────────────────────┘
                    │ ALL traffic            │ No traffic (standby)
                    ▼                        ▼
           ┌──────────────────┐       ┌──────────────────┐
           │  us-east-1       │       │  eu-west-1       │
           │  (ACTIVE)        │       │  (PASSIVE)       │
           │  ALB + EC2 ASG   │       │  ALB + EC2 ASG   │
           │  RDS Primary     │       │  RDS Read Replica│
           │  ElastiCache     │       │  ElastiCache     │
           └──────────────────┘       └──────────────────┘
                    │                        ▲
                    │  Continuous            │ Promoted on failover
                    │  replication ──────────┘

Route 53 failover configuration:
Primary:  us-east-1 ALB — health check on /health endpoint
          If health check fails → Route 53 automatically routes to
secondary

Secondary: eu-west-1 ALB — health check on same endpoint
           Normally dormant — only activated on failover

RDS in Active-Passive:
├── Primary region: RDS Multi-AZ (local HA)
├── Cross-region: RDS Read Replica in passive region
│   Read Replica lag: typically < 1 minute
└── Failover: promote Read Replica to standalone → becomes writable
    Promotion time: ~5 minutes + application reconfiguration

Aurora Global Database (recommended over RDS Read Replica):
├── Primary cluster: writes + reads in active region
├── Secondary cluster: read-only in passive region
├── Replication lag: < 1 second (vs minutes for RDS Read Replica)
└── Failover: promote secondary cluster in ~1 minute (managed failover)
```

```
# Route 53 health check + failover routing
# Step 1: Create health check for primary
```

```
aws route53 create-health-check \
  --caller-reference "primary-$(date +%s)" \
  --health-check-config '{
    "IPAddress":       "1.2.3.4",
    "Port":            443,
    "Type":            "HTTPS",
    "ResourcePath":    "/health",
    "FullyQualifiedDomainName": "app.us-east-1.company.com",
    "RequestInterval": 10,
    "FailureThreshold": 2
  }'

# Step 2: Create failover DNS records
aws route53 change-resource-record-sets \
  --hosted-zone-id Z123 \
  --change-batch '{
    "Changes": [
      {
        "Action": "CREATE",
        "ResourceRecordSet": {
          "Name": "app.company.com",
          "Type": "A",
          "SetIdentifier": "primary",
          "Failover": "PRIMARY",
          "AliasTarget": {
            "HostedZoneId": "ALB-HZ-ID",
            "DNSName": "alb.us-east-1.amazonaws.com",
            "EvaluateTargetHealth": true
          },
          "HealthCheckId": "health-check-id-primary"
        }
      },
      {
        "Action": "CREATE",
        "ResourceRecordSet": {
          "Name": "app.company.com",
          "Type": "A",
          "SetIdentifier": "secondary",
          "Failover": "SECONDARY",
          "AliasTarget": {
            "HostedZoneId": "ALB-HZ-ID-EU",
            "DNSName": "alb.eu-west-1.amazonaws.com",
            "EvaluateTargetHealth": true
          }
        }
      }
    ]
  }'
```

```
Active-Passive trade-offs:
Pros:
```

```
├── Lower cost: secondary region runs minimal resources
├── Simpler data consistency: one write source at a time
└── Simpler application: no need for conflict resolution

Cons:
├── Failover time: DNS TTL (60s) + health check interval + propagation =
2-5 min downtime
├── Passive region: paying for idle resources (or cold start time to warm
up)
├── Users in passive region: always routed to active region → high latency
└── Failback complexity: returning to primary after recovery
```

🧩 Active-Active Architecture

```
Active-Active:
├── BOTH (or ALL) regions serve live traffic simultaneously
├── Route 53 distributes traffic across regions (latency, weighted, or
geolocation)
├── Each region can handle full load independently (no single point of
failure)
└── Failure of one region: other regions absorb traffic automatically

┌──────────────────────────────────────────────────────────────────────┐
│                              Route 53                                 │
│  Latency routing: user routed to NEAREST healthy region               │
└──────────────────────────────────────────────────────────────────────┘
           │ US users (~50%)          │ EU users (~50%)
           ▼                          ▼
   ┌──────────────────┐        ┌──────────────────┐
   │   us-east-1      │        │   eu-west-1      │
   │   (ACTIVE)       │◄──────►│   (ACTIVE)       │
   │   ALB + ASG      │        │   ALB + ASG      │
   │   App servers    │        │   App servers    │
   │   Local cache    │        │   Local cache    │
   └──────────────────┘        └──────────────────┘
           │                          │
           ▼                          ▼
   ┌──────────────────────────────────────────────┐
   │  Global Data Layer                           │
   │  DynamoDB Global Tables: replicated <1sec    │
   │  Aurora Global Database: replicated <1sec    │
   │  ElastiCache Global Datastore                │
   └──────────────────────────────────────────────┘

Traffic distribution options:
├── Latency-based: Route 53 routes user to lowest-latency region
│   → US user → us-east-1, EU user → eu-west-1 (fastest)
├── Geolocation: route by user's country/continent
│   → EU GDPR: EU users MUST go to EU region (compliance)
└── Weighted: explicit percentage split (50/50, 90/10, etc.)
```

```
   → Useful for gradual traffic shift during multi-region migration

Global Accelerator vs CloudFront — frequently confused. CloudFront caches
content at edges (static assets, cacheable responses). Global Accelerator
uses edge nodes only as entry points — all processing still happens in the
AWS region. Global Accelerator is correct for dynamic, stateful, non-
cacheable traffic.

Route 53 failover is limited by DNS TTL and client-side caching — even
with TTL=60s, real-world failover often takes 2-5 minutes due to recursive
resolver and client caching. Global Accelerator eliminates this variable
entirely.
```

🧩 Multi-Region Data Challenges

```
The core challenge of Active-Active: write conflicts

If User A in us-east-1 and User B in eu-west-1 BOTH update the same record
simultaneously, what happens?

DynamoDB Global Tables conflict resolution:
├── Last-writer-wins using timestamp
├── Each region has its own write endpoint
├── Replication lag: typically < 1 second
└── If conflict: highest timestamp wins (may lose one write)
   ⚠ Application must be designed around last-writer-wins semantics

Aurora Global Database:
├── ONE write region (primary cluster) — all writes go there
├── Other regions: read-only replicas (<1 second lag)
└── Not truly active-active for writes — active-active for reads only
   (writes from EU region must route to primary write region)
   True active-active writes = must use DynamoDB Global Tables

Solutions to write conflicts:
├── Route by ownership: user data owned by one region (sticky routing)
│   User account shard → always write to that user's "home" region
├── Optimistic locking: version numbers on records → reject stale writes
└── Conflict-free data types (CRDTs): data structures that merge
automatically
    counters, sets — no conflicts possible by design

Active-Active infrastructure patterns:

S3 Cross-Region Replication:
├── Buckets replicated bidirectionally between regions
├── Objects written in us-east-1 → replicated to eu-west-1
└── Objects written in eu-west-1 → replicated to us-east-1

Lambda@Edge / CloudFront Functions:
```

```
├── Run code at 400+ CloudFront edge locations globally
├── Authentication, request routing, A/B testing at edge
└── Sub-millisecond additional latency at point of origin
```

🧩 Active-Active vs Active-Passive — Decision Framework

```
┌──────────────────────────┬──────────────────┬──────────────────┐
│ Consideration            │ Active-Active    │ Active-Passive   │
├──────────────────────────┼──────────────────┼──────────────────┤
│ Uptime requirement       │ 99.999%+         │ 99.9%-99.99%     │
│ Global user base         │ ✅ Yes           │ ⚠ High latency   │
│ Write conflict complexity │ High            │ Low (one writer) │
│ Cost                     │ 2× operating     │ 1.2-1.5× (standby)│
│ RTO on region failure    │ ~0 (automatic)   │ 2-10 minutes     │
│ Data residency compliance │ ✅ Geolocation  │ ✅ Failover stays │
│ Architecture complexity  │ Very high        │ Medium           │
│ Suitable databases       │ DynamoDB GT      │ Aurora Global    │
│                          │ (last-writer)    │ (read replicas)  │
└──────────────────────────┴──────────────────┴──────────────────┘

Choose Active-Active when:
├── SLA requires < 1 minute RTO (application-level automatic failover)
├── Global user base needs < 50ms latency worldwide
└── Business can absorb 2× operating cost and higher complexity

Choose Active-Passive when:
├── Primary goal is DR compliance (RPO/RTO requirements are minutes, not
seconds)
├── Writes need strong consistency (one active write region)
└── Cost constraints make running dual active regions prohibitive
```

💬 Short Crisp Interview Answer

"Active-active runs live traffic in multiple regions simultaneously — Route 53 latency routing sends users to the nearest healthy region, and if one region fails, others absorb traffic within seconds with zero manual intervention. The hard problem is write consistency: DynamoDB Global Tables uses last-writer-wins with sub-second replication, and Aurora Global Database has one write region with read replicas everywhere. Active-passive keeps one region dark until needed — simpler consistency (one writer always), lower cost, but 2-10 minute failover time via Route 53 health check detection and DNS propagation. Aurora Global Database managed failover takes about 1 minute. The decision comes down to RTO requirement: if you need < 1 minute RTO automatically, you need active-active. If minutes are acceptable with some manual steps, active-passive is far simpler and cheaper."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
DNS TTL during failover	Low TTL (60s) reduces failover time but increases DNS query load significantly
Route 53 health check interval	10-second interval + 2 failures = 20 seconds before failover starts
Aurora Global failover is manual by default	Must trigger managed failover explicitly — not automatic on region failure
DynamoDB Global Tables last-writer-wins	Conflicting simultaneous writes in two regions = one write silently lost
Active-active cost	Running full workload in two regions = nearly 2× the cost plus data transfer
Failback complexity	Returning to original primary after recovery requires careful state reconciliation

---

12.2 Disaster Recovery Strategies — RTO/RPO, Pilot Light, Warm Standby

---

🟢 What It Is in Simple Terms

Disaster Recovery (DR) is your plan for recovering from catastrophic failures — entire region down, mass data corruption, ransomware attack. RTO (Recovery Time Objective) is how long you have to recover. RPO (Recovery Point Objective) is how much data loss is acceptable. Four DR strategies exist on a spectrum from cheapest-and-slowest to most expensive-and-fastest.

🧩 RTO and RPO Defined

```
RPO — Recovery Point Objective:
"How much data can we afford to LOSE?"
"How old can our recovery state be?"

RPO = maximum acceptable time between last backup and the disaster

Example:
├── RPO = 24 hours: acceptable to lose up to 24 hours of data
│   → Daily backups are sufficient
├── RPO = 1 hour:   can lose up to 1 hour of data
│   → Hourly backups or continuous replication
├── RPO = 0:        zero data loss acceptable
│   → Synchronous replication required (expensive)
└── RPO = 15 minutes: S3 cross-region replication, Aurora Global Database

RTO — Recovery Time Objective:
```

```
"How long can the system be DOWN during recovery?"
"When does business impact become unacceptable?"

RTO = maximum acceptable downtime from disaster to recovery

Example:
├── RTO = 24 hours: acceptable to be down for a day
│   → Manual restoration from backups
├── RTO = 4 hours:  must be operational within 4 hours
│   → Warm standby + manual failover
├── RTO = 15 minutes: must recover in 15 minutes
│   → Near-automated failover
└── RTO = < 1 minute: cannot tolerate meaningful downtime
    → Active-active multi-region deployment

The relationship:
Lower RTO + Lower RPO = better DR = much more expensive

Cost vs protection curve:
More data loss OK, longer downtime OK ←──────────────────────────→ Zero
loss, zero downtime
Backup & Restore    Pilot Light    Warm Standby    Multi-Site Active-
Active
    $                  $$               $$$                 $$$$
Hours-days RTO      Hours RTO       Minutes RTO         < 1min RTO
Hours-days RPO      Minutes RPO     Minutes RPO         Near-zero RPO
```

🧩 DR Strategy 1 — Backup and Restore

```
Backup and Restore = cheapest DR. No standby infrastructure.
                     Recover by restoring backups from scratch.

RTO: Hours to days (restore + restart takes time)
RPO: Hours (frequency of last backup)
Cost: Lowest ($)

Architecture:
├── Production: runs in us-east-1
├── Backups: S3 Cross-Region Replication to eu-west-1 bucket
│   All data: database dumps, EBS snapshots, AMIs
└── Disaster: provision new infrastructure in eu-west-1 from scratch
              restore from backups

Automated backup setup:
# RDS automated backups (0-35 day retention)
aws rds modify-db-instance \
  --db-instance-identifier prod-mysql \
  --backup-retention-period 35 \
  --backup-window "02:00-03:00" \
  --copy-tags-to-snapshot
```

```
# Cross-region backup copy (daily Lambda job)
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier arn:aws:rds:us-east-
1:123:snapshot:rds:prod-mysql-2024-01-15 \
  --target-db-snapshot-identifier prod-mysql-2024-01-15-eu \
  --destination-region eu-west-1 \
  --kms-key-id arn:aws:kms:eu-west-1:123:key/...

# S3 Cross-Region Replication for application data
aws s3api put-bucket-replication \
  --bucket prod-data-us-east-1 \
  --replication-configuration '{
    "Role": "arn:aws:iam::123:role/s3-replication",
    "Rules": [{
      "Status": "Enabled",
      "Filter": {},
      "Destination": {
        "Bucket": "arn:aws:s3:::prod-data-eu-west-1",
        "ReplicationTime": {
          "Status": "Enabled",
          "Time": {"Minutes": 15}   ← S3 RTC: 99.99% replicated within 15
min
        },
        "Metrics": {"Status": "Enabled"}
      }
    }]
  }'

# AMI copy for compute layer
aws ec2 copy-image \
  --name "prod-web-ami-2024-01-15" \
  --source-image-id ami-0abc123 \
  --source-region us-east-1 \
  --region eu-west-1

Recovery playbook (manual, validated quarterly):
1. Restore VPC + networking in eu-west-1 (from Terraform/CloudFormation)
2. Launch EC2 from latest copied AMI
3. Restore RDS from latest snapshot
4. Update DNS to point to eu-west-1 endpoints
5. Verify application health
Typical time: 4-8 hours
```

🧩 DR Strategy 2 — Pilot Light

```
Pilot Light = minimal standby infrastructure, scaled up on disaster.
              Critical components (database) running in DR region.
              Everything else turned off (EC2 stopped, ASG at 0).
```

```
Name origin: like a pilot light in a gas furnace — tiny flame always on,
             can ignite full flame when needed.

RTO: 1-4 hours (scale up compute + update DNS)
RPO: Minutes (continuous database replication running)
Cost: Low-Medium ($$)

Architecture:
                   Production (us-east-1)         DR (eu-west-1)
                   ______________________         ______________
Compute:           EC2 ASG (active)       →      EC2 ASG = 0 instances
(STOPPED)
Load Balancer:     ALB (serving traffic)  →      ALB (provisioned, no
targets)
Database:          RDS Primary (active)   →      RDS Read Replica (RUNNING
— small instance)
Cache:             ElastiCache (active)   →      Not running
DNS:               Route 53 → primary    →      Route 53 → secondary
(failover record)

Pilot light = database replication is ALWAYS running.
              Compute is OFF — must be started and scaled on disaster.

Recovery steps on disaster:
1. Scale up ASG in DR region: desired=10 (was 0)
2. Promote RDS Read Replica to standalone writable instance (~5 min)
3. Update application config (DB endpoint changed after promotion)
4. Warm up ElastiCache (or cold start)
5. Route 53 health check triggers DNS failover to DR region
6. Verify traffic and health
Typical RTO: 1-4 hours (mostly waiting for DB promotion + ASG scale-up)

Automation with SSM:
# DR runbook: SSM Automation document to orchestrate recovery
aws ssm start-automation-execution \
  --document-name "PilotLightFailover" \
  --parameters '{
    "TargetRegion":    ["eu-west-1"],
    "DesiredCapacity": ["10"],
    "DBIdentifier":    ["prod-mysql-replica"]
  }'
# Automation: promotes DB, scales ASG, updates Route 53 — ~1 hour fully
automated
```

🧩 DR Strategy 3 — Warm Standby

```
Warm Standby = reduced-scale replica of production ALWAYS RUNNING.
               Can handle some traffic immediately.
               Scaled up to full capacity on disaster.
```

```
RTO: Minutes (scale up, not start from zero)
RPO: Seconds to minutes (continuous replication running)
Cost: Medium-High ($$$)

Architecture:
              Production (us-east-1)              Warm Standby (eu-west-1)
              ______________________              ________________________
Compute:      EC2 ASG min=10, max=100    →       EC2 ASG min=2, max=100
              (full production scale)            (reduced scale — just
enough to validate)
Load Balancer: ALB (serving traffic)     →       ALB (serving small %
traffic or just health)
Database:      RDS Primary               →       Aurora Global DB
secondary (reads, < 1sec lag)
Cache:         ElastiCache cluster       →       ElastiCache smaller
cluster
DNS:           Route 53 → primary        →       Route 53 → secondary (or
small %)

DR region is RUNNING but at 1/5 to 1/10 of production scale.

Recovery steps on disaster:
1. Scale ASG from min=2 to desired=10 (~3 minutes)
2. Aurora Global Database: promote secondary cluster (~1 minute)
3. DNS: shift 100% traffic to DR region (~60 seconds propagation)
4. Monitor and scale as needed
Typical RTO: 5-15 minutes (mostly automated)

Route 53 weighted routing for gradual DR validation:
# Send 5% of traffic to DR region always (validates it works)
aws route53 change-resource-record-sets \
  --hosted-zone-id Z123 \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name":          "app.company.com",
        "Type":          "A",
        "SetIdentifier": "primary",
        "Weight":        950,
        "AliasTarget": {
          "DNSName":            "alb.us-east-1.amazonaws.com",
          "EvaluateTargetHealth": true
        }
      }
    }, {
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name":          "app.company.com",
        "Type":          "A",
        "SetIdentifier": "dr",
        "Weight":        50,
        "AliasTarget": {
          "DNSName":            "alb.eu-west-1.amazonaws.com",
```

```
            "EvaluateTargetHealth": true
          }
        }
      }]
    }'
  # 5% always flows through DR — proves recovery path is healthy
  continuously
```

🧩 DR Strategy 4 — Multi-Site Active-Active

```
(Covered in depth in 12.1 — summary here)

Multi-Site Active-Active = full production running in all regions
simultaneously.
                           Automatic failover with near-zero RTO and RPO.

RTO: < 1 minute (automatic, no human action)
RPO: Near-zero (synchronous or sub-second replication)
Cost: Highest ($$$$) — full production in every region

Architecture: Route 53 latency routing → full ASG + database in each
region
Data: DynamoDB Global Tables or Aurora Global Database
Failover: Route 53 health check removes failed region automatically
```

🧩 DR Strategy Summary

```
┌──────────────────┬──────────────┬──────────────┬──────────┬──────────────┐
│ Strategy         │ RTO          │ RPO          │ Cost     │ Complexity   │
├──────────────────┼──────────────┼──────────────┼──────────┼──────────────┤
│ Backup & Restore │ Hours-days   │ Hours        │ $        │ Low          │
│ Pilot Light      │ 1-4 hours    │ Minutes      │ $$       │ Medium       │
│ Warm Standby     │ 5-30 minutes │ Seconds-min  │ $$$      │ High         │
│ Active-Active    │ < 1 minute   │ Near-zero    │ $$$$     │ Very high    │
└──────────────────┴──────────────┴──────────────┴──────────┴──────────────┘

DR testing is mandatory:
├── Backup & Restore: test quarterly (restore from backup, verify data)
├── Pilot Light: test quarterly (execute full failover runbook)
├── Warm Standby: test monthly (failover + failback drill)
└── Active-Active: continuous validation (5% traffic flows through DR
always)

⚠ An untested DR plan is not a DR plan.
   AWS GameDay and Chaos Engineering validate DR under realistic
conditions.
   Most DR failures are discovered during the actual disaster.
```

💬 Short Crisp Interview Answer

"DR strategy selection is driven by RTO and RPO requirements. Backup and Restore is the cheapest: store backups in a second region, restore from scratch on disaster — RTO of hours, RPO of hours, appropriate when the business can tolerate multi-hour downtime. Pilot Light keeps the database continuously replicated (the 'always-on' flame) with compute stopped — RTO of 1-4 hours to scale up, RPO of minutes, reasonable for most non-critical systems. Warm Standby runs a scaled-down replica always live — scale up on disaster for RTO of 5-30 minutes, RPO of seconds. Multi-site Active-Active is the most expensive but achieves sub-minute RTO and near-zero RPO automatically. The critical point: RPO drives your replication strategy, RTO drives how much infrastructure you keep pre-provisioned. An untested DR plan is worthless — validate with quarterly failover drills."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
RDS promotion changes endpoint	After promoting a Read Replica, the DB endpoint changes — application configs must update
Untested DR plans fail	Most companies discover DR gaps during real disasters — test quarterly minimum
Aurora Global promotion is manual	Aurora Global Database failover must be manually triggered — not automatic on region failure
RPO ≠ backup frequency alone	RPO includes time to restore + replication lag — not just how often you back up
Warm standby reduced capacity gap	If DR region at 20% capacity and disaster hits at peak load, DR region is overwhelmed
Failback is often harder than failover	Returning to primary after recovery requires re-syncing data and avoiding split-brain

---

12.3 Well-Architected Framework — 6 Pillars

🟢 What It Is in Simple Terms

The AWS Well-Architected Framework is AWS's set of architectural best practices organized into six pillars. Every AWS architecture review starts here. In interviews, the framework is used to evaluate how you think about system design — not just "does it work?" but "is it operationally excellent, secure, reliable, performant, cost-optimized, and sustainable?"

🧩 The Six Pillars

```
1. Operational Excellence
2. Security
3. Reliability
4. Performance Efficiency
5. Cost Optimization
6. Sustainability (added in 2021)

AWS Well-Architected Tool:
├── Free tool in AWS Console
├── Answer questions about your workload
├── Generates report: HIGH/MEDIUM/LOW risk items per pillar
└── Provides improvement plan with prioritized recommendations
```

🧩 Pillar 1 — Operational Excellence

```
Core question: "Can you run and monitor the system to deliver business
value
               and continually improve processes?"

Key design principles:
├── Perform operations as code (not manual clicking)
├── Make frequent, small, reversible changes (not big-bang releases)
├── Refine operations procedures frequently (runbooks stay current)
├── Anticipate failure (pre-mortem, chaos engineering)
└── Learn from all operational failures (blameless post-mortems)

Key practices:
├── Infrastructure as Code: Terraform/CDK for ALL infrastructure
│   → Manual infrastructure cannot be reliably reproduced or audited
├── Automated deployments: CI/CD pipeline, never SSH to deploy
│   → Human error in deployments is the #1 cause of outages
├── Runbooks for common operations: documented, automated where possible
│   → SSM Automation runbooks for DR, patching, scaling
├── Observability: metrics, logs, traces for every service
│   → CloudWatch + X-Ray + custom dashboards
├── Event response automation: EventBridge + Lambda for auto-remediation
└── Post-mortems: blameless analysis of every significant incident

Metrics to track:
├── MTTR (Mean Time To Recover): how fast do you recover from incidents?
├── MTBF (Mean Time Between Failures): how often do failures occur?
├── Deployment frequency: how often can you safely deploy?
├── Change failure rate: % of deployments causing incidents
└── Lead time for changes: idea to production

AWS tools:
├── CodePipeline, CodeDeploy: CI/CD automation
```

```
├── CloudFormation, CDK, Terraform: IaC
├── CloudWatch Dashboards, Alarms: operational visibility
├── Systems Manager: runbook automation
└── AWS Config: configuration compliance and drift detection
```

🧩 Pillar 2 — Security

```
Core question: "Can you protect information, systems, and assets while
               delivering business value through risk assessment?"

Key design principles:
├── Implement a strong identity foundation (least privilege, no long-lived
keys)
├── Enable traceability (CloudTrail for every API call)
├── Apply security at all layers (not just perimeter)
├── Automate security best practices (SCPs, Config rules)
├── Protect data in transit and at rest (TLS, KMS encryption)
├── Keep people away from data (automated processing, no direct DB access)
└── Prepare for security events (runbooks, incident response automation)

Security in depth model (layered):
┌──────────────────────────────────────────┐
│ Layer 1: Edge                            │
│ WAF + Shield + CloudFront                │
├──────────────────────────────────────────┤
│ Layer 2: Network                         │
│ VPC Security Groups + NACLs              │
│ No public IPs on application tier        │
├──────────────────────────────────────────┤
│ Layer 3: Compute                         │
│ IMDSv2, SSM (no SSH), Inspector          │
├──────────────────────────────────────────┤
│ Layer 4: Application                     │
│ Cognito, API Gateway auth, JWT           │
├──────────────────────────────────────────┤
│ Layer 5: Data                            │
│ KMS encryption, Secrets Manager,         │
│ S3 Block Public Access, RDS encryption  │
└──────────────────────────────────────────┘

Key AWS security services per layer:
Edge:        WAF, Shield Advanced, CloudFront
Identity:    IAM (least privilege), Organizations SCPs
Detection:   GuardDuty, Security Hub, Inspector, Macie
Response:    EventBridge + Lambda auto-remediation
Compliance:  AWS Config, Control Tower guardrails
```

🧩 Pillar 3 — Reliability

```
Core question: "Can the workload perform its intended function correctly
               and consistently?"

Key design principles:
├── Automatically recover from failure (no manual intervention required)
├── Test recovery procedures (chaos engineering, GameDay)
├── Scale horizontally (distribute load, eliminate single points of
failure)
├── Stop guessing capacity (auto scaling, serverless)
└── Manage change through automation (IaC, GitOps)

Reliability patterns on AWS:

1. Multi-AZ for everything:
   RDS Multi-AZ, ALB across 3 AZs, ASG across 3 AZs
   Single AZ = single point of failure

2. Circuit breaker pattern:
   Service → Circuit Breaker → Dependency
   If dependency fails: circuit opens → fast fail → no cascading failure
   Implemented in: App Mesh service mesh, application code

3. Retry with exponential backoff + jitter:
   Don't retry immediately — wait, then retry, with randomized delay
   Prevents thundering herd when a dependency recovers

4. Health checks and auto-healing:
   ALB health checks → remove unhealthy instances
   ASG health checks → replace terminated instances
   Route 53 health checks → failover to secondary region

5. Graceful degradation:
   If recommendations service is down: show empty recommendations
   Don't fail the whole page — degrade gracefully

6. Bulkhead pattern:
   Isolate critical from non-critical workloads
   Separate ASG for checkout (critical) vs reporting (non-critical)
   Failure in reporting ASG doesn't consume checkout resources

AWS reliability services:
├── ELB: distribute load, health check, remove unhealthy
├── Auto Scaling: replace failed instances, adjust capacity
├── Route 53: DNS-level health checks and failover
├── Multi-AZ: synchronous replication within region
└── Aurora: 6-copy storage across 3 AZs, automated failover < 30 seconds
```

🧩 Pillar 4 — Performance Efficiency

```
Core question: "Can you use computing resources efficiently to meet
requirements
               and maintain that efficiency as demand changes?"

Key design principles:
├── Democratize advanced technologies (use managed services, not self-
managed)
├── Go global in minutes (deploy to multiple regions with IaC)
├── Use serverless architectures (no instance management)
├── Experiment more often (easy rollback with IaC)
└── Consider mechanical sympathy (choose right resource for the job)

Performance optimization areas:

Compute:
├── Right instance family for workload:
│   Compute-intensive: C family (c5, c6i)
│   Memory-intensive:  R family (r5, r6i)
│   General purpose:   M family (m5, m6i)
│   GPU/ML:            P family (p3, p4), Inf2
│   Storage-optimized: I family (i3, i4i)
└── Graviton3 (ARM): 40% better price/performance for most workloads

Database:
├── Cache frequently accessed data: ElastiCache (Redis/Memcached)
├── Read replicas: distribute read load
├── Use right database type:
│   Relational: RDS Aurora
│   Key-value:  DynamoDB
│   Time-series: Timestream
│   Search:     OpenSearch
└── Connection pooling: RDS Proxy (critical for Lambda → RDS)

Networking:
├── CloudFront: serve content from nearest edge (sub-10ms to users)
├── Global Accelerator: anycast IP, route to nearest AWS region
├── Placement groups: low-latency cluster within AZ (HPC)
└── Enhanced Networking: ENA (up to 100 Gbps), EFA (HPC RDMA)

Testing performance:
├── Load testing: Artillery, k6, AWS Distributed Load Testing
├── CloudWatch metrics: P50, P95, P99 latency (not just averages!)
└── X-Ray traces: identify which service/dependency is the bottleneck
```

🧩 Pillar 5 — Cost Optimization

```
Core question: "Can you avoid unnecessary costs?"

Key design principles:
```

```
├── Implement cloud financial management (tagging, ownership, budgets)
├── Adopt a consumption model (pay only for what you use)
├── Measure overall efficiency (cost per business unit: cost per
transaction)
├── Stop spending on undifferentiated heavy lifting (use managed services)
└── Analyze and attribute expenditure (cost allocation tags + showback)

Key cost optimization levers (covered in Category 11):
├── Committed use: Savings Plans + Reserved Instances (30-72% savings)
├── Right-sizing: Compute Optimizer recommendations
├── Spot Instances: 60-90% discount for flexible workloads
├── Storage tiers: S3 lifecycle + Intelligent-Tiering
├── Data transfer: VPC Endpoints, CloudFront, topology-aware routing
└── Delete waste: Trusted Advisor idle resource detection

Cost metrics:
├── Cost per API call: track cost efficiency of each endpoint
├── Cost per customer: unit economics of infrastructure
└── Cost per GB processed: data pipeline efficiency
```

🧩 Pillar 6 — Sustainability

```
Core question: "Can you minimize environmental impact of your workloads?"

Added in 2021 — the newest pillar.

Key design principles:
├── Understand your impact (measure power consumption, carbon footprint)
├── Establish sustainability goals (commit to reduction targets)
├── Maximize utilization (right-size, avoid idle resources)
├── Anticipate and adopt new efficient hardware (Graviton, latest
generations)
├── Use managed services (AWS optimizes large shared infrastructure)
└── Reduce downstream impact (minimize client-side processing)

Sustainability practices on AWS:
├── Right-sizing: idle instances waste power
├── Graviton instances: 60% less energy for same performance
├── Serverless: Lambda, Fargate — zero idle resource consumption
├── AWS Carbon Footprint Tool: track estimated carbon emissions
└── AWS Regions vary: regions powered by renewable energy have lower
impact

Sustainability is cost-correlated:
"Maximize utilization, minimize waste" = both green and cheap.
Running idle resources is expensive AND environmentally wasteful.
```

🧩 Using the Well-Architected Framework in Interviews

```
When asked "How would you design X?" — walk through the pillars:

"Our e-commerce checkout service handles $10M/day transactions..."

Operational Excellence:
"We'd deploy via CI/CD pipeline with automated testing, Infrastructure as
Code for all resources, automated runbooks for scaling and recovery."

Security:
"No public IPs on app tier, WAF in front of ALB, KMS-encrypted RDS,
Secrets Manager for credentials, CloudTrail for all API activity,
GuardDuty for threat detection."

Reliability:
"Multi-AZ RDS, ALB + ASG across 3 AZs, Route 53 health checks for
regional failover, circuit breakers between microservices,
autoscaling on CPU and custom metrics."

Performance Efficiency:
"ElastiCache for product catalog (read-heavy, rarely changes),
RDS Proxy for connection pooling, CloudFront for static assets,
R-family instances for session/cart data (memory-intensive)."

Cost Optimization:
"Savings Plans for baseline EC2, Spot for background jobs,
S3 Intelligent-Tiering for product images, Trusted Advisor weekly
reviews."

Sustainability:
"Graviton3 instances, right-sized ASG with scale-to-zero in off hours,
serverless Lambda for async order processing."
```

💬 Short Crisp Interview Answer

"The Well-Architected Framework organizes AWS best practices into six pillars. Operational Excellence: everything as code, automated deployments, blameless post-mortems, runbooks for every operation. Security: defense in depth with WAF at edge, no public IPs on app tier, KMS encryption everywhere, GuardDuty for detection, least-privilege IAM. Reliability: Multi-AZ everything, auto-healing via ASG and Route 53 health checks, circuit breakers between services, chaos engineering to validate. Performance Efficiency: right instance family per workload, ElastiCache for hot data, RDS Proxy for connection pooling, CloudFront for latency. Cost Optimization: Savings Plans for baseline, Spot for flexible work, right-sizing from Compute Optimizer, data transfer optimization. Sustainability: Graviton instances, serverless where possible, maximize utilization. I use this framework as a checklist when reviewing any architecture."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail

Gotcha	Detail
Pillar order ≠ priority	Pillars are not in priority order — all matter equally, trade-offs are workload-specific
Well-Architected Tool is free	The tool is free but answering questions honestly takes significant time investment
Security vs cost trade-off	WAF, Shield Advanced, VPC Endpoints all add cost — security has a price
Reliability vs performance trade-off	Multi-AZ adds ~2× database cost but eliminates single-point-of-failure
Sustainability ≠ just carbon	Sustainability also includes developer efficiency and operational burden

---

12.4 Event-Driven Architecture on AWS

🟢 What It Is in Simple Terms

Event-driven architecture (EDA) is a design pattern where services communicate by producing and consuming events — things that happened — rather than calling each other directly. Instead of Service A making an HTTP call to Service B, Service A emits "OrderCreated" event, and any service that cares (payments, inventory, notifications) reacts independently. This creates loose coupling, independent scalability, and natural resilience.

🔍 Why Event-Driven / What Problem It Solves

```
Synchronous request-response problems:
Order Service → HTTP → Payment Service → HTTP → Inventory → HTTP → Email

Problems:
├── Cascading failures: Email service down = order creation fails
├── Tight coupling: Order service must know about all downstream services
├── Scaling together: Email service slowdown slows down order creation
├── Difficult evolution: adding new consumer = code change in Order
service
└── Blast radius: one slow service timeouts entire chain

Event-driven solution:
Order Service → "OrderCreated" event → EventBridge/SQS/SNS

Each consumer subscribes independently:
├── Payment Service    → processes payment asynchronously
├── Inventory Service  → reserves stock asynchronously
├── Notification Svc   → sends confirmation email asynchronously
```

```
└── Analytics Service  → updates dashboard asynchronously

Benefits:
├── Decoupling: Order service doesn't know who consumes its events
├── Independent scaling: each consumer scales by its own queue depth
├── Independent failure: email service down ≠ order creation fails
├── Easy addition: new consumer subscribes without touching Order service
└── Natural audit trail: event log is the history of what happened
```

🧩 Core EDA Patterns on AWS

```
Pattern 1: Simple Event Notification (SNS)
Producer publishes one event → SNS → all subscribers notified
simultaneously
├── Best for: broadcast (one event, many consumers, no ordering needed)
└── Example: "UserRegistered" → email, welcome sequence, analytics

Pattern 2: Work Queue (SQS)
Producer sends tasks → SQS → consumers process at their own pace
├── Best for: task distribution, load leveling, retry handling
└── Example: image resize jobs distributed across worker pool

Pattern 3: Fan-out (SNS + SQS)
Producer → SNS → multiple SQS queues → independent consumers
├── Best for: one event, multiple independent processing pipelines
└── Example: "OrderCreated" → [payments queue, inventory queue, email
queue]

Pattern 4: Event Streaming (Kinesis / MSK)
Producer → stream → multiple consumers read independently, replay possible
├── Best for: high-volume ordered events, multiple consumers, replay
└── Example: clickstream, IoT telemetry, audit log

Pattern 5: Event Bus with Routing (EventBridge)
Producers → EventBridge → rules route by content to specific consumers
├── Best for: complex routing based on event content, AWS service events
└── Example: GuardDuty HIGH severity → Lambda remediation only

Pattern 6: Choreography (pure EDA)
Services react to events without central orchestration
├── Each service does its work and emits new events
└── Example: OrderCreated → Payment processed → InventoryReserved →
OrderShipped

Pattern 7: Orchestration (Step Functions)
Central orchestrator tells each service what to do and handles failures
├── Best for: complex workflows with error handling, compensation
└── Example: multi-step order workflow with rollback on payment failure
```

🧩 Full Event-Driven Order System on AWS

```
Complete architecture: e-commerce order processing

┌──────────────────────────────────────────────────────────────────┐
│                      Order API (API Gateway + Lambda)            │
│                      POST /orders → create order record          │
└──────────────────────────────────────────────────────────────────┘
                              │ Publish "OrderCreated" event
                              ▼
                        ┌──────────────┐
                        │ EventBridge  │
                        │ Custom Bus   │
                        └──────────────┘
                              │
              ┌───────────────┼───────────────┐
              ▼               ▼               ▼
      ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
      │  SQS Queue   │  │  SQS Queue   │  │  SQS Queue   │
      │  payments    │  │  inventory   │  │  notifications│
      └──────────────┘  └──────────────┘  └──────────────┘
             │                 │                 │
             ▼                 ▼                 ▼
      ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
      │  Lambda      │  │  Lambda      │  │  Lambda      │
      │  Payment     │  │  Inventory   │  │  Email       │
      │  Processor   │  │  Updater     │  │  Sender      │
      └──────────────┘  └──────────────┘  └──────────────┘
             │ "PaymentProcessed" or "PaymentFailed"
             ▼
      ┌──────────────┐
      │ EventBridge  │
      └──────────────┘
             │
   ┌─────────┼─────────┐
   ▼         ▼         ▼
Order    Step Fn   Fulfillment
Update   (retry)  (if success)
```

```python
# Lambda: publish event to EventBridge after order creation
import boto3, json
from datetime import datetime

events = boto3.client('events')

def handler(event, context):
    # Create order in DynamoDB (not shown)
    order_id = create_order(event)

    # Publish event — don't call payment/inventory directly
    events.put_events(Entries=[{
```

```
        'Source':     'myapp.orders',
        'DetailType': 'OrderCreated',
        'EventBusName': 'prod-orders-bus',
        'Detail': json.dumps({
            'orderId':    order_id,
            'customerId': event['customerId'],
            'items':      event['items'],
            'totalAmount':event['totalAmount'],
            'currency':   'USD',
            'createdAt':  datetime.utcnow().isoformat()
        })
    }])

    # Return to API immediately — processing is now async
    return {'statusCode': 202, 'body': json.dumps({'orderId': order_id})}

# Status 202 Accepted — not 200 OK (processing continues asynchronously)
```

🧩 Event Schema Management

```
Events are contracts between services.
If producer changes event format → all consumers break.

Schema Registry (EventBridge + Glue Schema Registry):
├── Discover schemas from EventBridge events automatically
├── Validate events against schema before publishing
├── Schema versioning: v1, v2 (backward/forward compatible)
└── Generate code bindings (TypeScript, Python, Java) from schema

Schema evolution strategies:
├── Additive changes (safe): add new optional fields to existing schema
│   Old consumers: ignore new fields → still work
│   New consumers: read new fields → get new functionality
│
└── Breaking changes (dangerous): remove/rename required fields
    Strategy: version the event type
    v1: {"type": "OrderCreated.v1", "orderId": "..."}
    v2: {"type": "OrderCreated.v2", "orderId": "...", "currency": "..."}
    Both consumers exist during migration window
    Decommission v1 after all consumers migrated

Dead Letter Queue for events:
EventBridge DLQ:
├── Rule target fails repeatedly → event goes to SQS DLQ
└── Inspect + replay once consumer is fixed

SQS consumer DLQ:
├── Lambda processing fails maxReceiveCount times → DLQ
└── Alert + investigate + replay with start-message-move-task
```

🧩 Choreography vs Orchestration

```
CHOREOGRAPHY (pure event-driven):
├── No central coordinator — each service reacts to events independently
├── Services only know about events, not about other services
└── Natural audit trail: sequence of events = history of order lifecycle

Choreography flow:
OrderService: "OrderCreated"
→ PaymentService reacts: processes payment → "PaymentProcessed"
→ InventoryService reacts: reserves stock → "StockReserved"
→ ShippingService reacts: creates shipment → "OrderShipped"
→ NotificationService reacts: sends email

Pros:
├── Maximum decoupling — services don't call each other
└── Scales independently — each step can scale by its own queue depth

Cons:
├── Debugging: hard to trace the flow across services (need distributed
tracing)
├── Error handling: compensation (undo) logic spread across multiple
services
└── No single view: "what is the current state of order X?" is hard

ORCHESTRATION (Step Functions):
├── Central workflow coordinator explicitly calls each service in order
├── Handles failures, retries, compensation (undo) centrally
└── Provides visual workflow view + per-step execution history

Orchestration flow (Step Functions):
Order Workflow:
1. Reserve inventory → success? Continue | failure? Go to step 5
2. Process payment → success? Continue | failure? Go to step 4
3. Create shipment → success? Complete
4. Release inventory reservation (compensation)
5. Notify customer of failure → Complete

Pros:
├── Error handling and compensation in one place
├── Visual workflow: see exactly where each order is
└── Easy to add steps, change logic, add retries

Cons:
├── Central coordinator = potential bottleneck
└── Services must be callable (synchronous or via SQS/SNS)

When to use which:
├── Choreography: loosely coupled, independently scalable, simple flows
└── Orchestration: complex error handling, compensation needed, audit
trail required
```

```
    Use Step Functions Express for high-throughput (100K/sec), Standard
for long-running
```

💬 Short Crisp Interview Answer

"Event-driven architecture decouples producers from consumers through asynchronous events — instead of Service A calling Service B directly, A publishes 'OrderCreated' and B subscribes to act on it. This means B's failure doesn't prevent A from processing, each service scales independently by its own queue depth, and adding new consumers requires no changes to the producer. On AWS the main patterns are: SNS for broadcast, SQS for work queues and buffering, EventBridge for content-based routing and AWS service events, and Kinesis for ordered high-volume streams with replay. Choreography has services react to events independently — maximum decoupling, but complex compensation logic. Orchestration with Step Functions has a central coordinator — easier error handling and visual workflow, but adds coupling to the orchestrator. In practice I use both: EventBridge + SQS fan-out for the happy path, Step Functions for complex flows with compensation."

⚠️ Tricky Edge Cases / Gotchas

Gotcha	Detail
Events are contracts	Changing event schema breaks consumers — always version breaking changes
202 not 200 for async	Return HTTP 202 Accepted for async operations, not 200 OK
At-least-once delivery	All AWS event services deliver at-least-once — consumers must be idempotent
Distributed tracing required	Without X-Ray correlation IDs, debugging cross-service event flows is nearly impossible
Event ordering is hard	Only Kinesis and SQS FIFO guarantee ordering — EventBridge and standard SQS do not
Eventual consistency	Event-driven systems are eventually consistent — services may briefly have stale state

---

12.5 Zero-Downtime Deployments — Blue/Green, Canary on AWS

🟢 What It Is in Simple Terms

Zero-downtime deployment strategies let you release new versions of software without any service interruption. Instead of stopping the old version and starting the new one (causing downtime), you gradually

shift traffic from old to new while validating the new version's health at each step. If something goes wrong, you roll back by shifting traffic back — instantly.

🔍 Why It Exists

```
Naive deployment (causes downtime):
1. Stop all instances of v1
2. Deploy v2 to all instances
3. Start v2

Problems:
├── Users during step 1-3: service unavailable (downtime)
├── Bug in v2: all users impacted before you notice
└── Rollback: stop v2, restart v1 (another round of downtime)

Zero-downtime philosophy:
├── Traffic shifting: gradually move users from old to new version
├── Health validation: confirm new version is healthy before full cutover
└── Instant rollback: switch traffic back to old version in seconds
```

🧩 Deployment Strategy 1 — In-Place (Rolling)

```
Rolling Deployment = replace instances gradually, N at a time

Steps:
1. Take 25% of instances out of rotation (deregister from ALB)
2. Deploy v2 to those 25% instances
3. Register back into ALB → validate health
4. Repeat for remaining 75% in batches

┌──────────────────────────────────────────────────────┐
│  ALB                                                 │
│  Step 1: [v1][v1][v1][v1] → [--][v1][v1][v1]   │
│  Step 2: [--][v1][v1][v1] → [v2][v1][v1][v1]   │
│  Step 3: [v2][--][v1][v1] → [v2][v2][v1][v1]   │
│  Step 4: [v2][v2][--][v1] → [v2][v2][v2][v1]   │
│  Step 5: [v2][v2][v2][--] → [v2][v2][v2][v2]   │
└──────────────────────────────────────────────────────┘

AWS CodeDeploy rolling:
aws deploy create-deployment \
  --application-name MyApp \
  --deployment-config-name CodeDeployDefault.HalfAtATime \
  --deployment-group-name prod-web

Config options:
├── OneAtATime:    1 instance at a time (slow, safest)
├── HalfAtATime:   50% of fleet simultaneously
```

```
├── AllAtOnce:     all instances simultaneously (fastest, most risky)
└── Custom:        e.g., 25% of instances, wait 5 minutes, check alarms

Pros:
├── No extra infrastructure cost (reuse existing instances)
└── Gradual rollout (detect issues before full cutover)

Cons:
├── During deployment: both v1 and v2 serving traffic simultaneously
│   Application must be backward-compatible with mixed versions
├── Rollback: must re-deploy v1 to all instances (slow)
└── Partial availability: capacity reduced during deployment

⚠ Post-Rollback Errors — Why They Happen:

Even after kubectl rollout undo completes successfully,
users may see errors for 30-60 seconds because:

1. ALB deregistration delay (default 300s):
   → Reduce to 30s for fast APIs

2. New (old-version) pods not yet healthy:
   → Tune health check: interval=10s, threshold=2
   → Healthy in 20 seconds

3. Terminating pods not handling SIGTERM gracefully:
   → App must catch SIGTERM, finish in-flight requests
   → Add preStop hook: sleep 5s before SIGTERM

Real-world rule:
"kubectl rollout undo success ≠ zero errors immediately"
Always monitor error rates for 2-3 minutes after rollback.
```

🧩 Deployment Strategy 2 — Blue/Green

```
Blue/Green = two IDENTICAL environments, switch traffic instantly

"Blue" = current production (v1)
"Green" = new version (v2) — deployed and tested before traffic switch

┌──────────────────────────────────────────────────────────────────┐
│                        ALB or Route 53                           │
│                                                                  │
│  Before:  100% → [Blue  (v1)]  [Green (v2)] ← 0%               │
│           (blue is live)  (green being tested/warmed up)         │
│                                                                  │
│  Cutover: 0%  → [Blue  (v1)]  [Green (v2)] ← 100%              │
│           (instantaneous traffic switch)                         │
│                                                                  │
│  Rollback: 100% → [Blue (v1)] [Green (v2)] ← 0%                 │
```

```
│           (switch back in seconds if issue detected)          │
└──────────────────────────────────────────────────────────────┘

Blue/Green with EC2 + ALB:
# Step 1: Deploy v2 to new Target Group (green)
aws elbv2 create-target-group \
  --name "prod-web-green" \
  --protocol HTTP --port 80 \
  --vpc-id vpc-123

# Step 2: Register new v2 EC2 instances to green target group
# Step 3: Test green target group (internal health check, smoke tests)

# Step 4: Switch ALB to route to green target group
aws elbv2 modify-listener \
  --listener-arn arn:aws:elasticloadbalancing:...:listener/... \
  --default-actions '[{
    "Type": "forward",
    "TargetGroupArn": "arn:...prod-web-green"
  }]'
# Instant: ALL traffic now going to green (v2)

# Step 5: Monitor for 15-30 minutes
# Step 6a: Success → deregister and terminate blue instances
# Step 6b: Issue → revert listener back to blue instantly

Blue/Green with CodeDeploy:
aws deploy create-deployment-group \
  --application-name MyApp \
  --deployment-group-name prod-bg \
  --deployment-style
'{"deploymentType":"BLUE_GREEN","deploymentOption":"WITH_TRAFFIC_CONTROL"}
' \
  --blue-green-deployment-configuration '{
    "terminateBlueInstancesOnDeploymentSuccess": {
      "action": "TERMINATE",
      "terminationWaitTimeInMinutes": 30
    },
    "deploymentReadyOption": {
      "actionOnTimeout": "CONTINUE_DEPLOYMENT",
      "waitTimeInMinutes": 0
    }
  }' \
  --load-balancer-info '{
    "targetGroupInfoList": [{"name":"prod-web-green"}]
  }'
```

```
Blue/Green for ECS (native support):
ECS + CodeDeploy natively supports blue/green:
├── Task definition v2 deployed to new tasks (green)
├── ALB points to blue (v1) target group
```

```
├── Green tasks warmed up and health-checked
├── ALB shifted to green with configurable traffic shift
└── Blue tasks kept alive for configurable period then terminated

Blue/Green for Lambda:
└── Lambda Aliases: prod alias → points to function version

Pros:
├── Instant rollback: switch listener back to blue in < 1 second
├── Full green environment tested before ANY production traffic
├── Zero capacity reduction during deployment
└── Easy canary testing: shift small % to green first

Cons:
├── 2× infrastructure cost during deployment window (pay for both blue +
green)
└── More complex for stateful services (session state, database
migrations)
```

🧩 Deployment Strategy 3 — Canary

```
Canary = send small % of traffic to new version, expand gradually

Origin: "canary in a coal mine" — small group of users are the canary.
If something goes wrong, only canary users are affected — not everyone.

┌──────────────────────────────────────────────────────────────────┐
│  ALB/Route 53 weighted routing                                   │
│                                                                  │
│  Step 1:  95% → v1  |  5% → v2  ← test with real traffic   │
│  Step 2:  80% → v1  | 20% → v2  ← looks good, expand       │
│  Step 3:  50% → v1  | 50% → v2  ← monitoring closely       │
│  Step 4:   0% → v1  |100% → v2  ← full cutover             │
│                                                                 │
│  If alarm fires at any step → revert to 0% → v2             |
└──────────────────────────────────────────────────────────────────┘

Canary with Route 53 weighted routing:
# Initial canary: 5% to new version
aws route53 change-resource-record-sets \
  --hosted-zone-id Z123 \
  --change-batch '{
    "Changes": [
      {
        "Action": "UPSERT",
        "ResourceRecordSet": {
          "Name": "app.company.com",
          "Type": "A",
          "SetIdentifier": "stable",
          "Weight": 95,
```

```
            "AliasTarget": {"DNSName": "alb-v1.us-east-1.amazonaws.com"}
          }
        },
        {
          "Action": "UPSERT",
          "ResourceRecordSet": {
            "Name": "app.company.com",
            "Type": "A",
            "SetIdentifier": "canary",
            "Weight": 5,
            "AliasTarget": {"DNSName": "alb-v2.us-east-1.amazonaws.com"}
          }
        }
      ]
    }'
```

```
Canary with CodeDeploy + automatic rollback:
aws deploy create-deployment-group \
  --deployment-group-name prod-canary \
  --deployment-config-name CodeDeployDefault.LambdaCanary10Percent5Minutes
\
  --auto-rollback-configuration '{
    "enabled": true,
    "events": [
      "DEPLOYMENT_FAILURE",
      "DEPLOYMENT_STOP_ON_ALARM",
      "DEPLOYMENT_STOP_ON_REQUEST"
    ]
  }' \
  --alarm-configuration '{
    "enabled": true,
    "alarms": [
      {"name": "prod-error-rate-high"},
      {"name": "prod-p99-latency-high"}
    ]
  }'

CodeDeploy built-in canary configs for Lambda:
├── LambdaCanary10Percent5Minutes:  10% canary → 5 min → 100%
├── LambdaCanary10Percent10Minutes: 10% → 10 min → 100%
├── LambdaCanary10Percent15Minutes: 10% → 15 min → 100%
├── LambdaLinear10PercentEvery1Minute: 10% more every 1 minute
└── LambdaLinear10PercentEvery2Minutes: 10% more every 2 minutes

Auto-rollback on CloudWatch Alarm:
├── CodeDeploy monitors CloudWatch alarms during deployment
├── If alarm triggers (error rate spike): automatic rollback initiated
├── Rollback: Lambda alias pointed back to previous version
└── No human intervention required — detected and rolled back
automatically
```

```
Canary for Lambda aliases:
aws lambda create-alias \
  --function-name prod-processor \
  --name prod \
  --function-version 1 \
  --routing-config '{"AdditionalVersionWeights":{"2": 0.05}}'
# prod alias: 95% to version 1, 5% to version 2
# Update weight as canary validates:
aws lambda update-alias \
  --function-name prod-processor \
  --name prod \
  --routing-config '{"AdditionalVersionWeights":{"2": 0.5}}'
# Now 50/50
# Full cutover:
aws lambda update-alias \
  --function-name prod-processor \
  --name prod \
  --function-version 2 \
  --routing-config '{}'
# 100% to version 2
```

🧩 Deployment on EKS — Rolling + Blue/Green

```
Kubernetes Rolling Update (default):
# Update deployment image — Kubernetes handles rolling automatically
kubectl set image deployment/web-app web=myapp:v2

# Kubernetes rolling update strategy:
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge:       25%   # max extra pods above desired during update
      maxUnavailable: 0%    # never reduce below desired capacity
# With maxUnavailable=0: always have full capacity during update (safe)
# With maxUnavailable=25%: allow 25% capacity reduction (faster update)

Blue/Green on EKS with weighted routing:
# Two Deployments: web-app-blue (v1) and web-app-green (v2)
# AWS Load Balancer Controller: weighted target groups

# Step 1: deploy green
kubectl apply -f web-app-green.yaml

# Step 2: split traffic 90/10
kubectl annotate service web-app \
  alb.ingress.kubernetes.io/target-group-weights='[
    {"name":"web-app-blue","weight":90},
    {"name":"web-app-green","weight":10}
  ]'
```

```
# Step 3: monitor, then full cutover
kubectl annotate service web-app \
  alb.ingress.kubernetes.io/target-group-weights='[
    {"name":"web-app-blue","weight":0},
    {"name":"web-app-green","weight":100}
  ]'

# Step 4: delete blue deployment after validation period
kubectl delete deployment web-app-blue

Flagger (Argo Rollouts / Flagger):
├── GitOps-compatible canary controller for Kubernetes
├── Automatically progresses canary based on metrics
(Prometheus/CloudWatch)
├── Automatic rollback if metrics degrade
└── Integrates with: AWS App Mesh, NGINX, ALB ingress
```

🧩 Database Migrations in Zero-Downtime Deployments

```
Database migrations are the hardest part of zero-downtime deployments.
Cannot deploy v2 code that depends on a v2 schema while v1 code is still
running.

Expand-Contract Pattern (also called Branch-by-Abstraction):

Phase 1: EXPAND (backward-compatible schema change)
├── Add new column (nullable): ALTER TABLE orders ADD COLUMN currency
VARCHAR(3)
├── Deploy v2 code that WRITES to both old + new column
└── Old code still works (new column is nullable, ignored by old code)

Phase 2: MIGRATE (backfill data)
├── Run background job: UPDATE orders SET currency='USD' WHERE currency IS
NULL
└── Verify all rows have currency value

Phase 3: CONTRACT (remove old column)
├── Deploy v3 code that ONLY reads new column
├── Old code no longer running (deployment complete)
└── DROP COLUMN old_field (safe now — no code reads it)

Why this works:
├── Phase 1 deploy: both v1 and v2 code can run simultaneously (backward
compatible)
├── No migration required before deployment
└── Rollback at Phase 1: v1 code still works (new column has no data yet)

Database migration tools:
├── Flyway: SQL-based migrations, version-controlled
```

```
├── Liquibase: XML/YAML schema change management
└── Django/Rails/Laravel: built-in ORM migration frameworks

RDS Blue/Green Deployments (AWS native):
├── AWS creates a staging (green) RDS instance
├── Synchronizes with production continuously
├── Allows testing schema changes on staging
└── Switchover: ~1-2 minutes of brief write pause, then green becomes
primary
```

🧩 Choosing the Right Strategy

```
┌──────────────────────┬──────────────────┬──────────────────┬──────────────
──┐
│ Factor               │ Rolling          │ Blue/Green       │ Canary
│
├──────────────────────┼──────────────────┼──────────────────┼──────────────
──┤
│ Cost during deploy   │ No extra cost    │ 2× cost          │ Minimal extra
│
│ Rollback speed       │ Slow (re-deploy) │ Instant          │ Instant
│
│ Traffic split        │ No               │ Hard cut         │ Gradual
│
│ Risk surface         │ All users once   │ All users once   │ Small % first
│
│ Complexity           │ Low              │ Medium           │ Medium-High
│
│ State management     │ Medium           │ Hard             │ Medium
│
│ Best for             │ Internal tools   │ APIs, services   │ High-stakes
│
│                      │ Low-traffic apps │ customer-facing  │ new features
│
└──────────────────────┴──────────────────┴──────────────────┴──────────────
──┘

Feature flags (additional pattern):
Deploy v2 code to all users with new feature disabled by default.
Toggle feature on for 1% users → 10% → 100% → via Launch Darkly / AWS
AppConfig.

Decouples code deployment from feature release:
├── Code deploys can happen anytime (risky new code behind flag = off)
└── Feature releases can happen anytime (business controls rollout)

AWS AppConfig:
├── Feature flags and configuration values
├── Gradual rollout with bake time and rollback
└── Integrates with Lambda, ECS, EC2 via SDK
```

💬 Short Crisp Interview Answer

"Zero-downtime deployments shift traffic gradually rather than replacing all at once. Rolling updates replace instances in batches — cheapest but during deployment both versions serve traffic simultaneously, so the app must be backward-compatible. Blue/green runs two complete environments and switches traffic instantly — the best rollback story (sub-second revert) but doubles infrastructure cost during deployment window. Canary routes 5-10% of traffic to the new version first, monitors CloudWatch alarms for error rate and latency, then either progresses to 100% or auto-rolls back if alarms fire — CodeDeploy natively supports this for Lambda and ECS. Database migrations are the hard part: use the expand-contract pattern — first deploy adds new columns (backward compatible with old code), second deploy stops writing to old columns, third deploy drops them. For EKS, set maxUnavailable=0 in the rolling update strategy so you never reduce capacity during a deployment."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Rolling: both versions live	App must handle mixed-version traffic during rolling — backward-compatible APIs required
Blue/Green: 2× cost	Full second environment running during deployment — plan for the cost
Canary: session affinity	Users may see different versions across requests if sessions aren't sticky — design for this
DB migrations before code	Schema changes must be backward-compatible before new code deploys — expand first
ALB deregistration delay	Default 300 seconds drain before deregistering instance — reduce for faster rolling deployments
Lambda alias routing	Lambda alias canary traffic split is not deterministic per user — same user can see both versions
EKS PodDisruptionBudget	Set PDB to prevent rolling update from removing too many pods during ASG scaling events

- **Default deregistration delay of 300 seconds causes post-cutover errors** — reduce to 30 seconds for APIs with short request durations. The default is designed for long-running requests and actively hurts fast blue/green switches.
- **Green instance warmup is separate from ALB health checks** — a green instance can pass /health checks while still being cold internally (empty caches, unwarmed connection pools). Always pre-warm before switching traffic.

🔗 Category 12 — Full Connections Map

```
ARCHITECTURE PATTERNS connects to:

Multi-Region Active-Active / Active-Passive
├── Route 53             → latency, geolocation, weighted, failover
routing
├── Aurora Global        → < 1 second cross-region replication
├── DynamoDB Global      → multi-master, last-writer-wins, sub-second sync
├── S3 CRR               → cross-region replication with S3 RTC (15-min
SLA)
├── CloudFront           → global edge for static content, Lambda@Edge
├── Global Accelerator   → anycast routing to nearest AWS region
└── ALB                  → regional load balancing within each active
region

Disaster Recovery
├── RDS                  → Read Replicas across regions, automated
snapshots
├── Aurora Global        → sub-second RPO, 1-minute RTO for DB failover
├── S3 Cross-Region      → backup storage with 99.999999999% durability
├── Route 53             → DNS failover with health checks
├── CloudEndure          → continuous block-level replication for lift-
and-shift DR
├── AWS Backup           → centralized backup across RDS, EBS, EFS,
DynamoDB
└── SSM Automation       → automated DR runbooks for orchestrated failover

Well-Architected Framework
├── Every AWS service    → WAF covers all services across all 6 pillars
├── Trusted Advisor      → automated pillar checks (cost, security,
reliability)
├── Security Hub         → security pillar detective controls
├── Config               → operational excellence + reliability compliance
├── Cost Explorer        → cost optimization pillar metrics
└── CloudWatch           → operational excellence observability

Event-Driven Architecture
├── EventBridge          → event bus, rules, cross-account routing
├── SQS                  → async decoupling, buffering, work queues
├── SNS                  → broadcast fan-out to multiple consumers
├── Kinesis              → ordered streams, replay, multiple consumers
├── Step Functions       → orchestration with error handling +
compensation
├── Lambda               → event consumers (triggered by all event
sources)
├── DynamoDB Streams     → change data capture → events
└── MSK                  → Kafka-native event streaming

Zero-Downtime Deployments
├── CodeDeploy           → rolling, blue/green, canary deployment
automation
├── ALB                  → target group shifting for blue/green
├── Route 53             → weighted routing for canary traffic shift
```

```
  ├── Lambda Aliases       → version routing for Lambda canary
  ├── ECS                  → native CodeDeploy blue/green with task
  replacement
  ├── CloudWatch Alarms    → auto-rollback trigger on error rate / latency
  spike
  ├── AppConfig            → feature flags for decoupled feature releases
  └── RDS Blue/Green       → managed database switchover for schema
  migrations
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
Active-Active RTO	Near-zero (automatic, Route 53 removes failed region within seconds)
Active-Passive RTO	2-10 minutes (health check + DNS propagation + service warm-up)
Active-Active write challenge	DynamoDB Global Tables = last-writer-wins. Aurora Global = one write region
Aurora Global failover	~1 minute. Must be manually triggered — not automatic on region failure
DynamoDB Global Tables	Sub-second replication. Last-writer-wins conflict resolution
Route 53 failover TTL	Low TTL (60s) reduces failover time but increases DNS query load
RPO definition	Maximum acceptable data loss — drives replication strategy
RTO definition	Maximum acceptable downtime — drives pre-provisioning strategy
Backup & Restore RTO/RPO	Hours-days / Hours. Cheapest. No standby infrastructure
Pilot Light RTO/RPO	1-4 hours / Minutes. DB always on. Compute stopped
Warm Standby RTO/RPO	5-30 minutes / Seconds. Scaled-down version always running
Active-Active RTO/RPO	< 1 minute / Near-zero. Full infrastructure in all regions
Aurora Global promotion	Manual — must trigger promoted failover explicitly
RDS Read Replica promotion	Endpoint changes after promotion — app configs must update
DR testing requirement	Untested DR = no DR. Test quarterly minimum
Well-Architected pillars	Operational Excellence, Security, Reliability, Performance, Cost, Sustainability
Operational Excellence key	Everything as code, automated deployments, blameless post-mortems

Topic	Key Fact
Security key	Defense in depth, no public IPs on app tier, KMS everywhere, GuardDuty
Reliability key	Multi-AZ everything, auto-healing ASG, circuit breakers, chaos testing
Performance key	Right instance family, ElastiCache, RDS Proxy, CloudFront
Cost Optimization key	Savings Plans baseline, Spot for flexible, right-sizing, data transfer optimization
Sustainability key	Graviton instances, serverless, maximize utilization, minimize waste
EDA core benefit	Decoupling — producer doesn't know consumers, each scales independently
Choreography vs Orchestration	Choreography = services react to events. Orchestration = Step Functions coordinates
At-least-once in EDA	All AWS event services deliver at-least-once — consumers must be idempotent
Event schema changes	Breaking changes = version event type. Additive changes = safe (backward compatible)
202 vs 200 for async	Return 202 Accepted for async event-driven operations
Rolling deployment cost	No extra cost. But mixed versions live simultaneously — app must be backward-compatible
Blue/Green rollback	Sub-second: switch ALB listener back to blue target group
Blue/Green cost	2× infrastructure during deployment window
Canary auto-rollback	CodeDeploy monitors CloudWatch alarms — fires alarm → automatic rollback
Lambda canary	Lambda aliases with AdditionalVersionWeights — 5% to v2
CodeDeploy canary configs	LambdaCanary10Percent5Minutes (10% for 5 min then 100%) and others built-in
DB migration strategy	Expand-Contract: add column (backward compat) → backfill → remove old column
Rolling maxUnavailable=0	Never reduces capacity during update — safe for production
EKS PodDisruptionBudget	Prevents rolling update from removing too many pods simultaneously
ALB deregistration delay	Default 300 seconds drain — reduce to 30s for faster deployments if traffic is short-lived
Feature flags vs deployment	AppConfig decouples code deploy from feature release — code ships hidden, feature toggles on

Category 12: Architecture Patterns — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation

🎓 Complete Interview Preparation — All 12 Categories

```
You now have the complete AWS DevOps / SRE / Platform Engineer interview
guide:

Category 1:  AWS Core Fundamentals (IAM, Global Infrastructure, CLI,
Organizations)
Category 2:  Compute (EC2, ASG, ELB, Spot, Launch Templates)
Category 3:  Networking & VPC (VPC, SG/NACL, TGW, Endpoints, Route 53,
CloudFront)
Category 4:  Storage (S3, EBS, EFS, FSx, Storage Gateway)
Category 5:  Containers & Serverless (ECS, EKS, Lambda, ECR, Fargate)
Category 6:  Databases (RDS, Aurora, DynamoDB, ElastiCache, Redshift)
Category 7:  Observability (CloudWatch, X-Ray, ADOT, Container/Lambda
Insights)
Category 8:  Security (IAM deep, KMS, Secrets Manager, GuardDuty, WAF,
Config)
Category 9:  Messaging (SQS, SNS, EventBridge, Kinesis, MSK)
Category 10: Infrastructure & Automation (SSM, Terraform, CDK, Control
Tower, RAM)
Category 11: Cost Optimization (Cost Explorer, Savings Plans, Spot, S3
tiers, Data Transfer)
Category 12: Architecture Patterns (Multi-Region, DR, Well-Architected,
EDA, Deployments)

Total: 75 topics across 12 categories — complete senior engineer
preparation.
```